Product Feedback - Tools Used in Meridian

Feedback on every tool, API, and SDK used, as required by the hackathon submission.

1. MCP TypeScript SDK (`@modelcontextprotocol/sdk` 1.32.1)

Used for: the self-hosted Meridian MCP server - Streamable HTTP transport, session management, tool registration with Zod schemas, resource registration for the MCP App HTML, and server-initiated notifications.

What worked well:
- The Streamable HTTP transport handled everything correctly once wired: initialize/session IDs, 202-on-notification, DELETE teardown, and the standalone GET SSE stream all behaved per spec.
- `registerTool` + Zod schemas give real input validation for free - malformed tool calls fail loudly with useful parse errors.
- The `notifications/message` logging channel is exactly the right primitive for streaming agent progress to a live UI.

What needs work:
- **Capability gating is silent:** `notifications/message` requires the `logging` capability to be declared; without it the notification is silently dropped rather than throwing at registration time. This cost us a debugging cycle (we fixed it by declaring `logging: {}`).
- **`registerTool` callback shape surprised us:** with a non-empty schema the callback receives parsed args first, but with `{}` the shape was ambiguous in early attempts; a doc example showing both paths would help.
- **Input schema must be a Zod shape (not raw JSON Schema)** - the error message says so, but the docs don't lead with this; we initially passed JSON Schema and hit a runtime error.
-(GET stream keep-alive defaults to 15s; during development it looked like the push was broken until we found the keep-alive comments. A shorter default or a log line would help.)

Onboarding (zero to hello world): moderate. The TypeScript quickstart is good, but real-world Streamable HTTP server wiring (single endpoint, CORS preflight for browser clients, session header handling) required reading transport source code. Working overall integration on day one would not have happened without the SDK being open-source.

Would we build with it again? Yes. It is the official implementation and the wire-level correctness is excellent; the friction is all in discoverability of the finer requirements (capabilities and schema forms).

2. MCP spec (2025-11-25) + MCP Apps extension (SEP-1865 / 2026-01-26)

Used for: the overall server architecture and the `ui://` resource pattern that renders the day canvas inside the chat client.

What worked well:
- The spec text for Streamable HTTP is unambiguous and testable - session IDs, version header, resumability semantics are all precisely defined, which let us verify our server against a plain curl script.
- The MCP Apps "tool declares a ui:// resource, host renders sandboxed iframe" pattern is a genuinely great idea - it turned our tool output into a real interactive product surface.

What needs work:
- There is no official reference *host* beyond a basic example, so implementing the simulator side (iframe postMessage protocol) required reading the ext-apps source and the spec side by side.
- The MCP Apps docs are spread across the extension site, the spec repo, and blog posts; a single canonical quickstart for "server app + custom host" would cut hours.

Onboarding: good for the core spec, patchy for Apps.

Would we build with it again? Yes - MCP Apps is the most exciting part of the MCP ecosystem right now.

3. Featherless AI (OpenAI-compatible inference, Llama-3.3-70B)

Used for: the agentic planning brain - the primary provider behind our function-calling loop, via the OpenAI-compatible `/v1/chat/completions` endpoint with `tools` / `tool_calls`.

What worked well:
- Genuinely OpenAI-compatible: the standard `tools` array + `tool_choice: "auto"` just worked on `unsloth/Llama-3.3-70B-Instruct` - multi-step plans (time check -> search -> canvas write) executed with zero prompt gymnastics.
- HUGE model catalog (~22k models) with per-model pricing metadata exposed directly on `/v1/models`, so selecting by context length and class is a quick script, not docs spelunking.
- Fast, and free credits let us iterate through dozens of full agent loops without quota anxiety.

What needs work:
- Model discovery is the weak spot: 22k ids with no capability tags (which support tool calling? which are instruct-tuned?). We picked by name heuristics and verified with a test call; a `capabilities` field on the model object would help a lot.
- Gated vs open is in the list, but there is no documented permission-error shape - a failing gated call returns a bare error.

Onboarding: excellent - key in, first `/v1/models` call in under a minute, working tool-calling completion in five.

Would we build with it again? Yes - it is now our default provider.

4. Google Gemini (`@google/genai` 2.28.0, gemini-3.8-flash)

Used for: the automatic fallback brain - Gemini's native function-calling loop over the shared tool declarations, transparently used if the primary provider errors.

What worked well:
- Native tool-calling is reliable and fast on 3.8-flash; multi-step plans (time check -> search -> canvas write) worked without prompt gymnastics.

What needs work:
- Constructing the client with a missing key throws at import time in some setups; lazy init is safer for libraries.
- Function-call results require the exact `functionResponse` part shape; a typed helper for building the convo array would reduce boilerplate.

Onboarding: excellent - the AI Studio key flow is the fastest free start available. One wrinkle: older model IDs (`gemini-2.0-flash`, `gemini-2.5-flash`) return 404 for new keys; the error payload helpfully names the successor (`gemini-3.8-flash`), and we wish every API surfaced the replacement model that directly.

Would we build with it again? Yes.

5. Tavily Search API

Used for: the `search_web` tool (weather, factual lookups during planning).

What worked well: simple REST API, clean result shape, sensible free tier - integrated in minutes.

What needs work: rate-limit error bodies could be more descriptive.

Would we build with it again? Yes.

Friction Log

#	Task attempted	Steps taken	Expected vs actual	Severity	Workaround	Suggestion
1	Stream server-initiated notifications over GET stream	Declared `notifications/message` sends, opened GET stream	Expected frames on stream; got silence	High	Declared `logging: {}` capability	Fail fast at `notification()` when capability missing
2	Register tools with schemas	Passed JSON Schema objects to `registerTool`	Runtime error: "inputSchema must be a Zod schema or raw shape"	Medium	Converted to Zod shapes	Accept JSON

Schema directly or document conversion path |
| 3 | Browser simulator calling the server | POSTs from localhost:5173 | Preflight 405 - no OPTIONS handling | Medium | Added explicit OPTIONS + CORS headers with `mcp-session-id` exposure | Ship a tiny CORS helper for HTTP server transports |
| 4 | Debugging "silent" GET stream | Watched stream during pushes | Nothing appeared for ~15s | Low | Set `keepAliveMs: 3000` | Document keep-alive default prominently |